

SOFTWARE DESIGN

Review

- Software Requirements
- Requirements Engineering Process

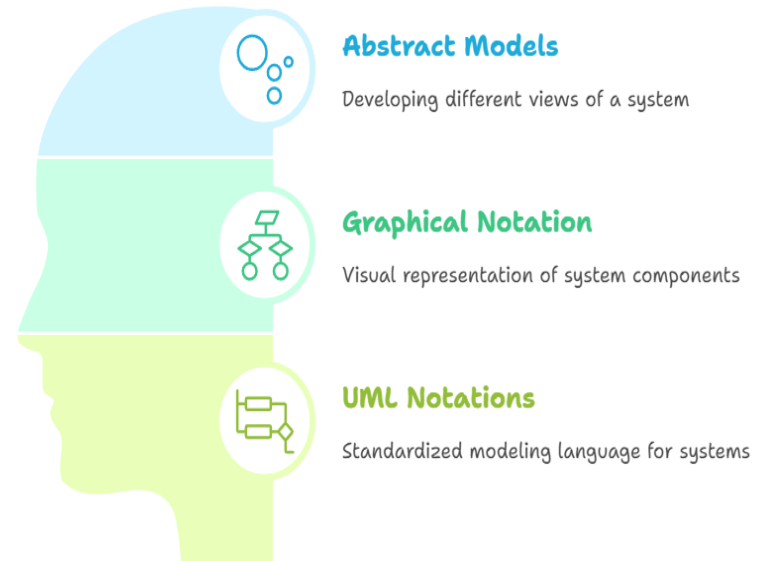
Outline

- ANALYSIS PHASE (OBJECT ORIENTED DESIGN)
- Functional Modeling
 - Use case
 - Diagram
 - Description

System Modeling

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling has generally come to mean representing the system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).

Understanding System Modeling



System Modeling

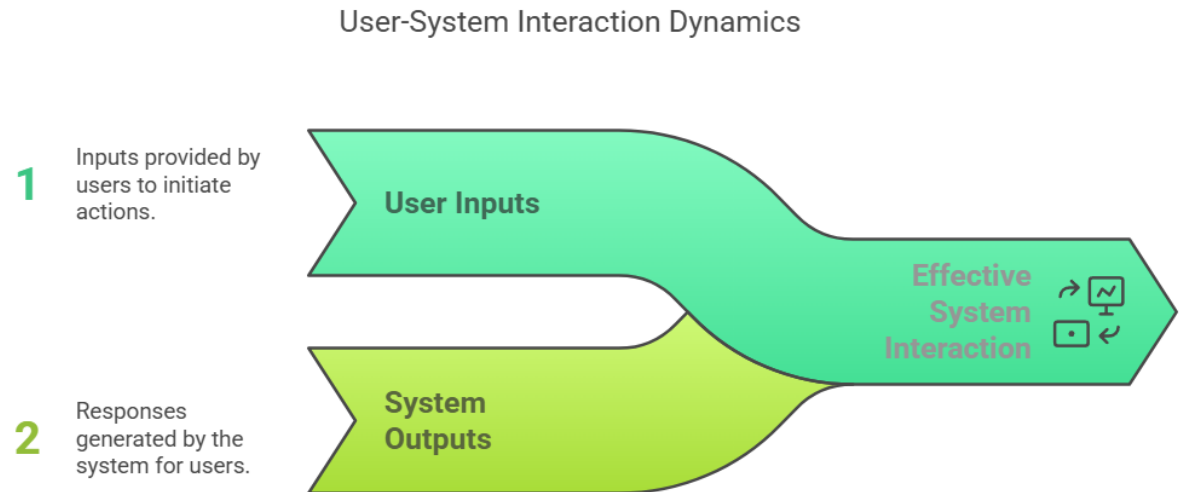
- Models are used
 - during the requirements engineering process to help derive the requirements for a system,
 - during the design process to describe the system to engineers implementing the system and
 - after implementation to document the system's structure and operation.

Analysis Models

- At an early stage in the specification of a system, we need to decide on the system boundaries.
 - This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment.
- We need to decide that automated support for some business processes should be implemented but
 - others should be manual processes or supported by different systems.

Modeling System Interactions

- All systems involve interaction of some kind.
 - user interaction, which involves user inputs and outputs, interaction between the system being developed
- Modeling user interaction is important as it helps to identify user requirements.



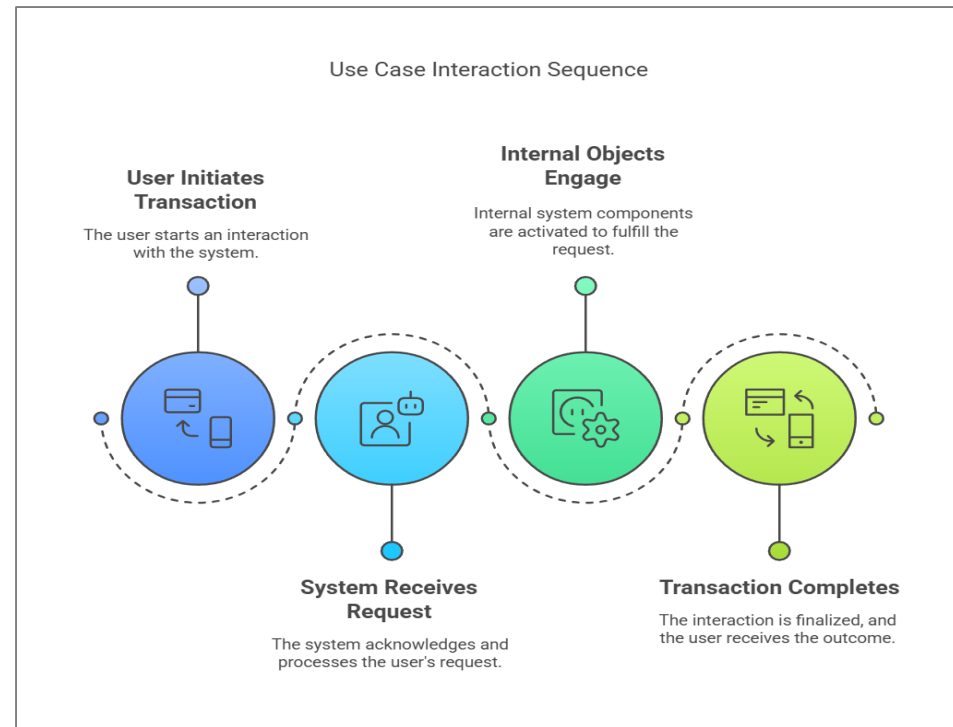
Modeling System interactions

- For modeling system interactions we can use
 - Use case modeling,
 - which is mostly used to model interactions between a system and external actors (users or other systems).

USE CASE MODELING

Use Case

- A use case is a set of scenarios that describe an interaction between a user and a system.
- Use case model is a representation of sequence of transactions initiated by the user (actor) from outside the system.
- In the process, the transaction uses internal objects to take the transaction to completion.



Use Case

- Use case defines and describes what happens in the system in logical order, termed as system behavior.
- They represent the flows of events that the user triggers. The user/actor is anything that initiates or triggers the action in the system.

Use Case and System Behavior Sequence



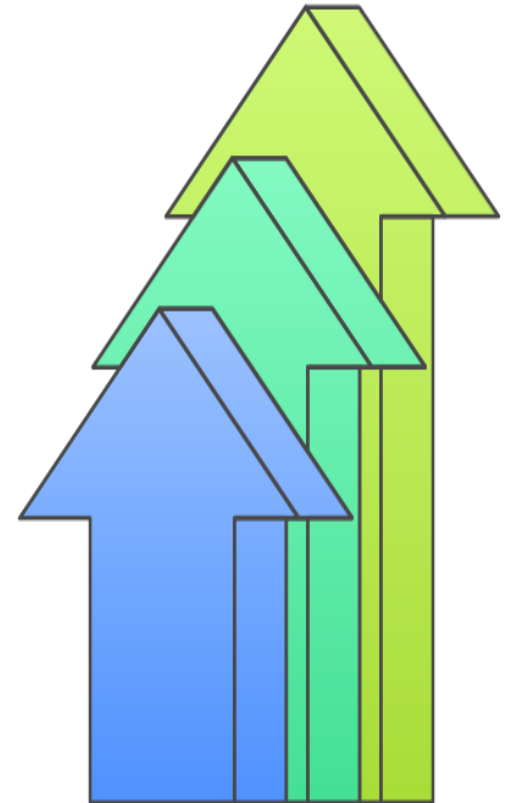
The user or actor triggers an action in the system.



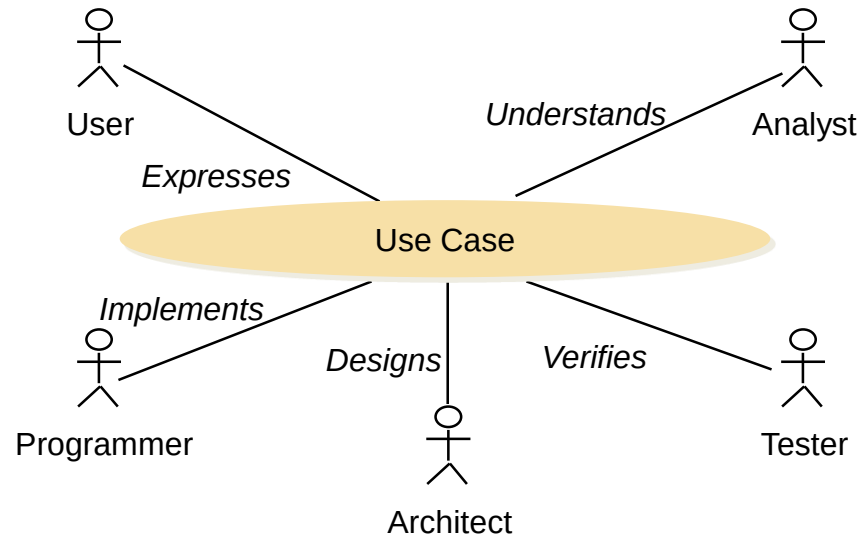
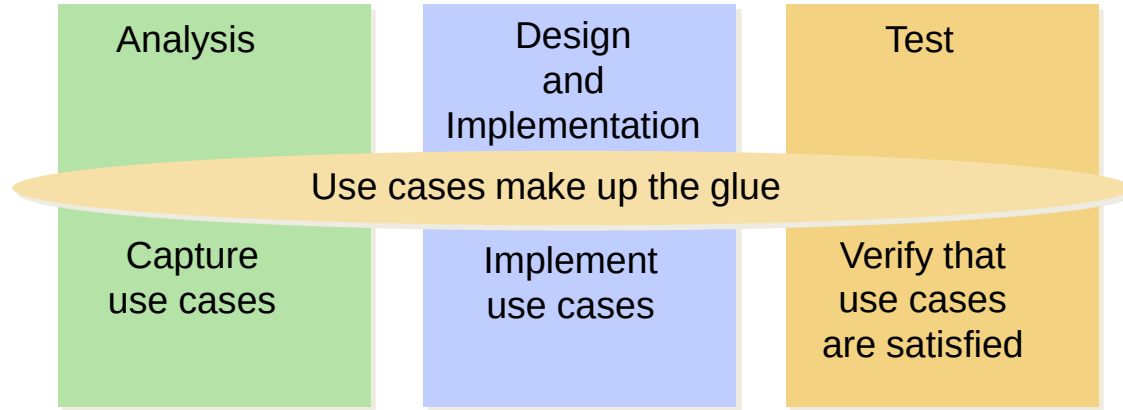
The system processes the action initiated by the user.

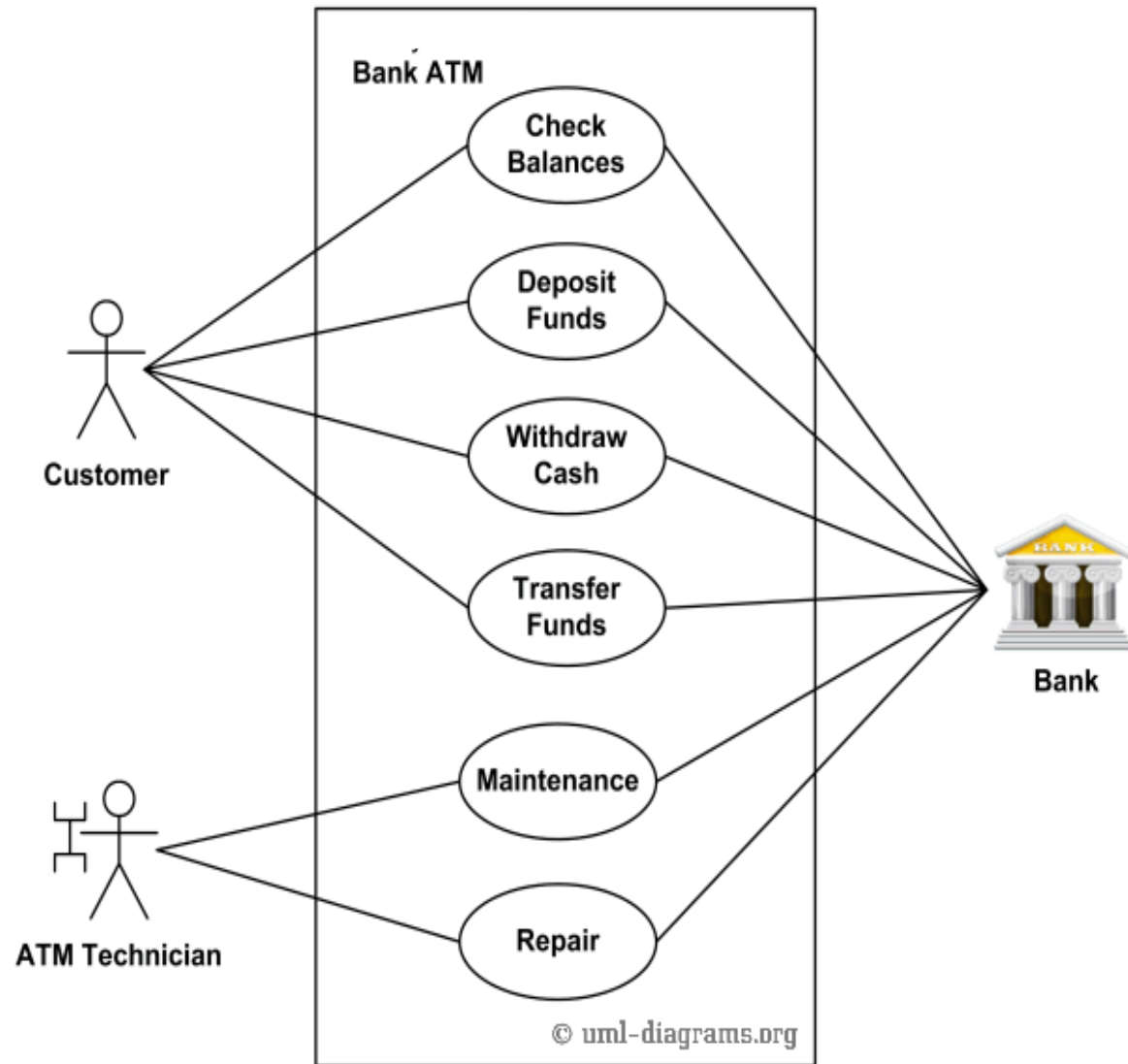


The sequence of actions is described as system behavior.



The Scope of Use Cases





Basic Concepts

- Use cases are not inherently object-oriented
 - An external (user) view of the system
 - Intended for modelling the dialog between the users and the system

Use Cases

- Use case diagram.
- Detailed use case document.

Use Case Diagram

- UML artifact to represent the overall system specifications.
- A way to conceive and illustrate the use cases.
- Shows system boundary, main functionalities, the external entities which can interact with the system.

Example: Ticket Reservation System

- The passenger has a prior knowledge of the reservation and ticketing system.
- The passenger arrives at the ticket counter and interacts with the clerk first through an enquiry
- Passenger then follows the process of form filling, tendering, payment and collecting the tickets.
- Passenger accepts the ticket or leaves the counter.

Notations Used

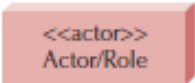
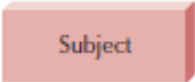
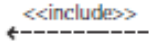
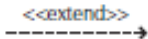
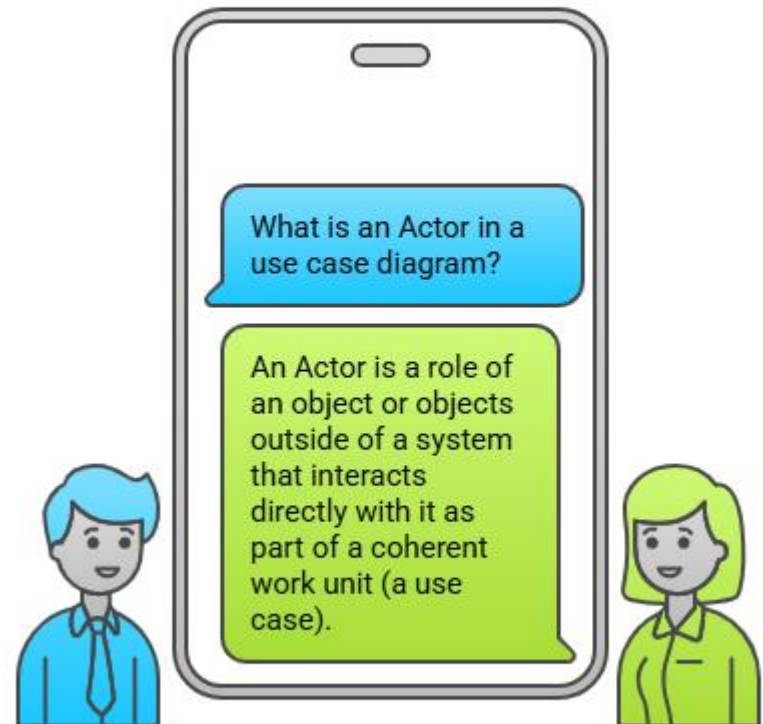
An Actor: ■ Is a person or system that derives benefit from and is external to the subject ■ Is depicted as either a stick figure (default) or if a non-human actor is involved, as a rectangle with <<actor>> in it (alternative) ■ Is labeled with its role ■ Can be associated with other actors using a specialization/superclass association, denoted by an arrow with a hollow arrowhead ■ Are placed outside the subject boundary	 Actor/Role 
A Use Case: ■ Represents a major piece of system functionality ■ Can extend another use case ■ Can include another use case ■ Is placed inside the system boundary ■ Is labeled with a descriptive verb-noun phrase	
A Subject Boundary: ■ Includes the name of the subject inside or on top ■ Represents the scope of the subject, e.g., a system or an individual business process	
An Association Relationship: ■ Links an actor with the use case(s) with which it interacts	
An Include Relationship: ■ Represents the inclusion of the functionality of one use case within another ■ The arrow is drawn from the base use case to the included use case	
An Extend Relationship: ■ Represents the extension of the use case to include optional behavior ■ The arrow is drawn from the extension use case to the base use case	
A Generalization Relationship: ■ Represents a specialized use case to a more generalized one ■ The arrow is drawn from the specialized use case to the base use case	

FIGURE 6-6 Syntax for Use Case Diagram

Actor

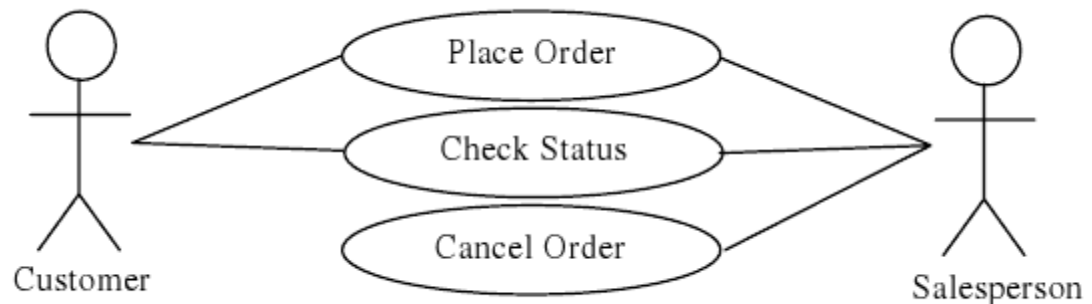
- An Actor is a role of an object or objects outside of a system that interacts directly with it as part of a coherent work unit (a use case)
- One physical object (or class) may play several different roles and be modeled by several actors
- Example actors for an Airline Reservation system
 - Airline administrators (fare/schedule setting)
 - Travel Agent
 - Airline Reservations Agent
 - Check-in Agents at Airport
 - Gate Agent at Airport

Understanding Actors in Use Case Diagrams



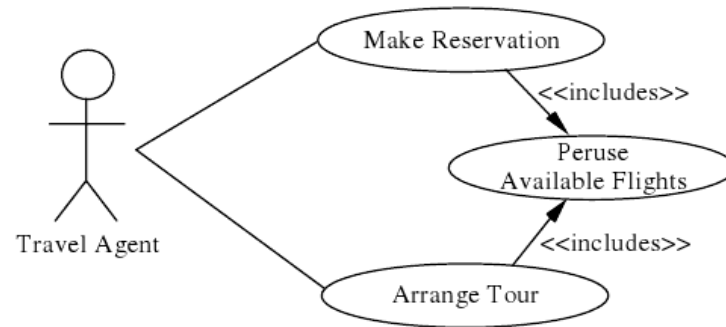
Use Case

- A Use Case captures some actor-visible function
- Achieves some discrete (business-level) goal for that actor
- May be read, write, or read-modify-write in nature
- Notation

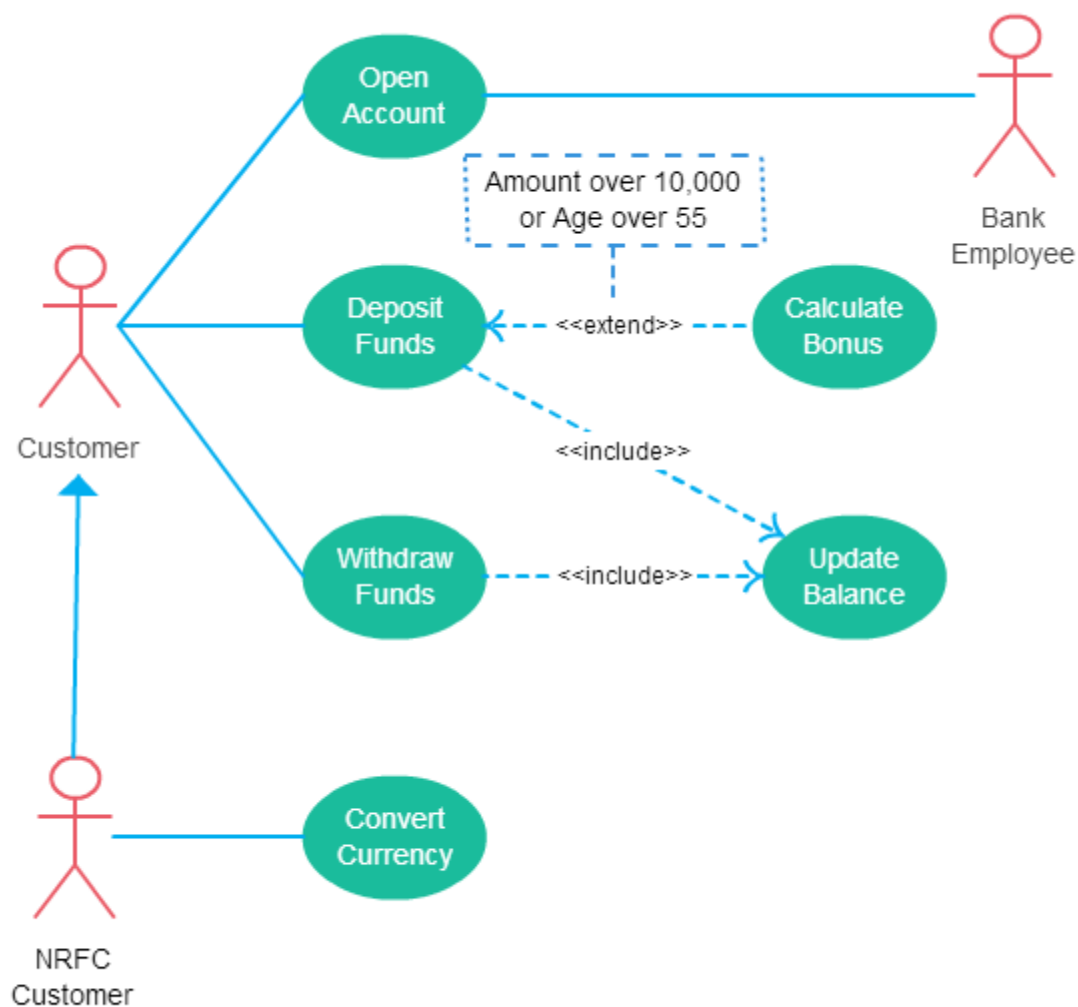


Associations: <<includes>>

- When a use case is depicted as using the functionality of another use case in a diagram.
- An include relationship is depicted with a directed arrow having a dotted shaft. The tip of the arrowhead points to the parent use case and the child use case is connected at the base of the arrow.
- Example



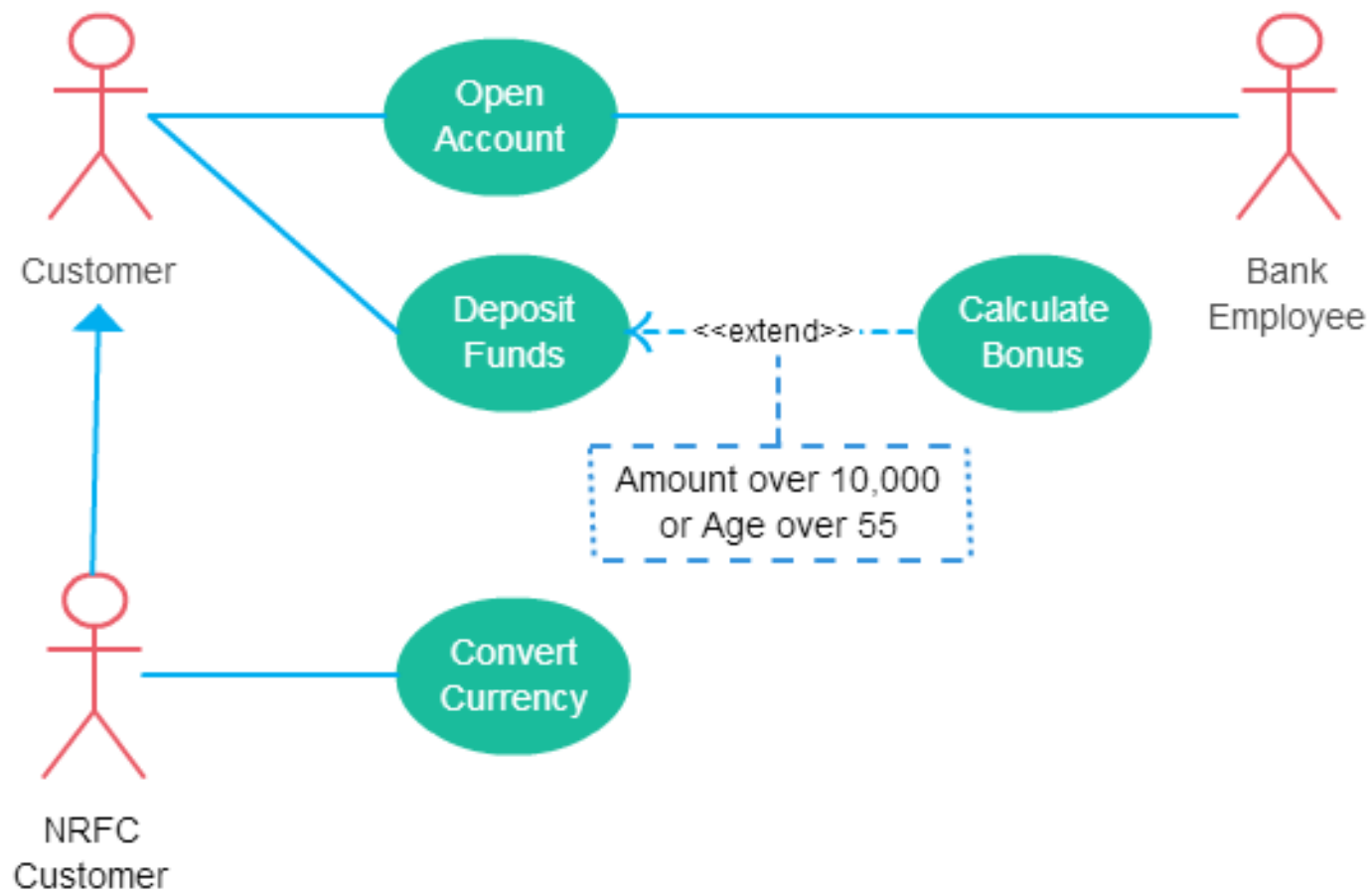
- Make Reservation and Arrange Tour both depend on Peruse /examine Available Flights
- Note that the arrows go from the dependent use cases
- Typically used when the same unit of functionality is part of more than one use case
- The base use cases are, in a sense, incomplete without the included use case



Includes is usually used to model common behavior

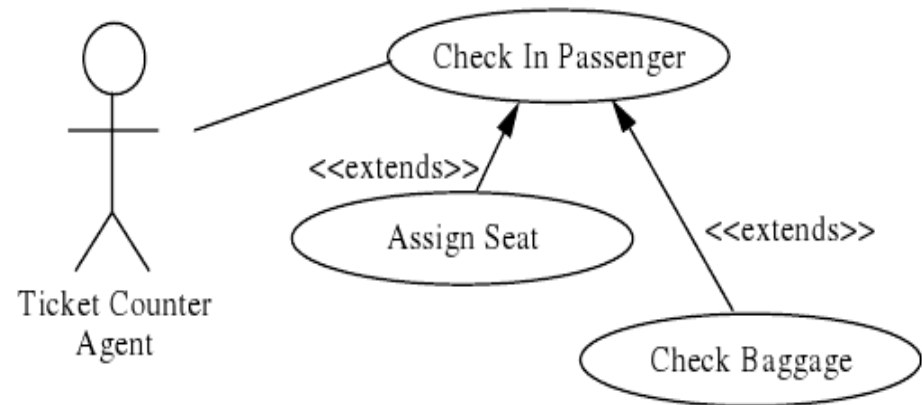
Associations: <<extends>>

- In an *extend* relationship between two use cases, the child use case adds to the existing functionality and characteristics of the parent use case.
- An extend relationship is depicted with a directed arrow having a dotted shaft, similar to the include relationship.
- The tip of the arrowhead points to the parent use case and the child use case is connected at the base of the arrow. The stereotype "<<extend>>" identifies the relationship as an extend relationship



Extend relationship in use case diagrams

- Example:



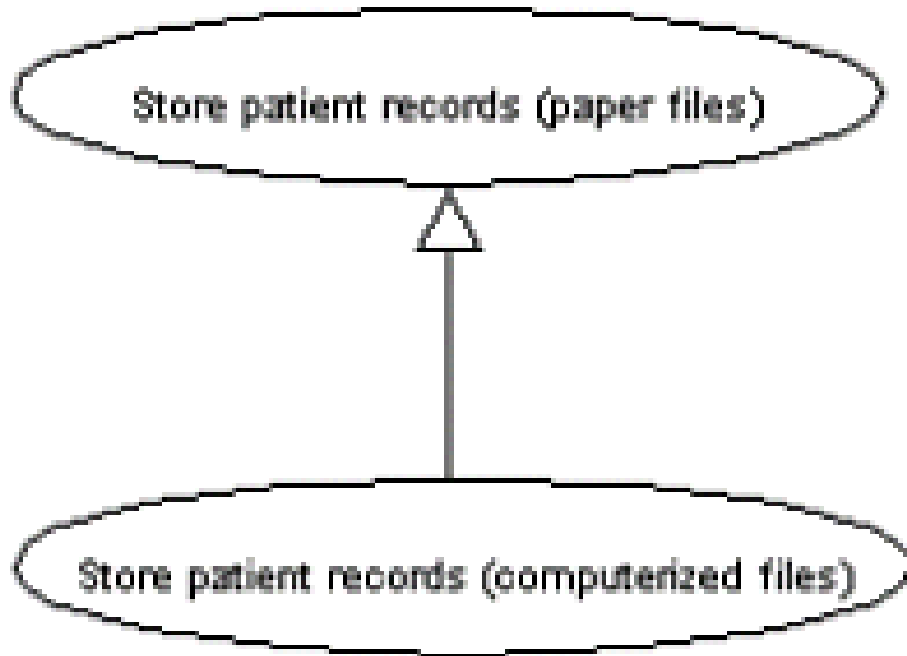
- Assign Seat and Check Baggage both depend on Check In Passenger
- Note that the arrows go from the dependent use cases
- Typically used when there are important, *optional* variations on the basic theme of the base use case
- The base use case is complete in and of itself

Associations: Generalizations:

- A generalization relationship is also a parent-child relationship between use cases.
- The child use case in the generalization relationship has the underlying business process meaning, but is an enhancement of the parent use case.
- In a use case diagram, generalization is shown as a directed arrow with a triangle arrowhead.
- The child use case is connected at the base of the arrow. The tip of the arrow is connected to the parent use case.

Associations

- Generalization Relationship



- Arrow directed from parent use case to child use case

Difference between Extend Relationship and Generalization Relationship

- On the face of it, both generalizations and extends appear to be more or less similar. But there is a subtle difference between a generalization relationship and an extend relationship.
 - a generalization relationship between use cases:
 - the parent use case can be replaced by the child use case without breaking the business flow.
 - an extend relationship between use cases:
 - that the child use case enhances the functionality of the parent use case into a specialized functionality. The parent use case in an extend relationship cannot be replaced by the child use case.

Example: Ticket Reservation System

- The passenger has a prior knowledge of the reservation and ticketing system.
- The passenger arrives at the ticket counter and interacts with the clerk first through an enquiry
- Passenger then follows the process of form filling, tendering, payment and collecting the tickets.
- Passenger accepts the ticket or leaves the counter.

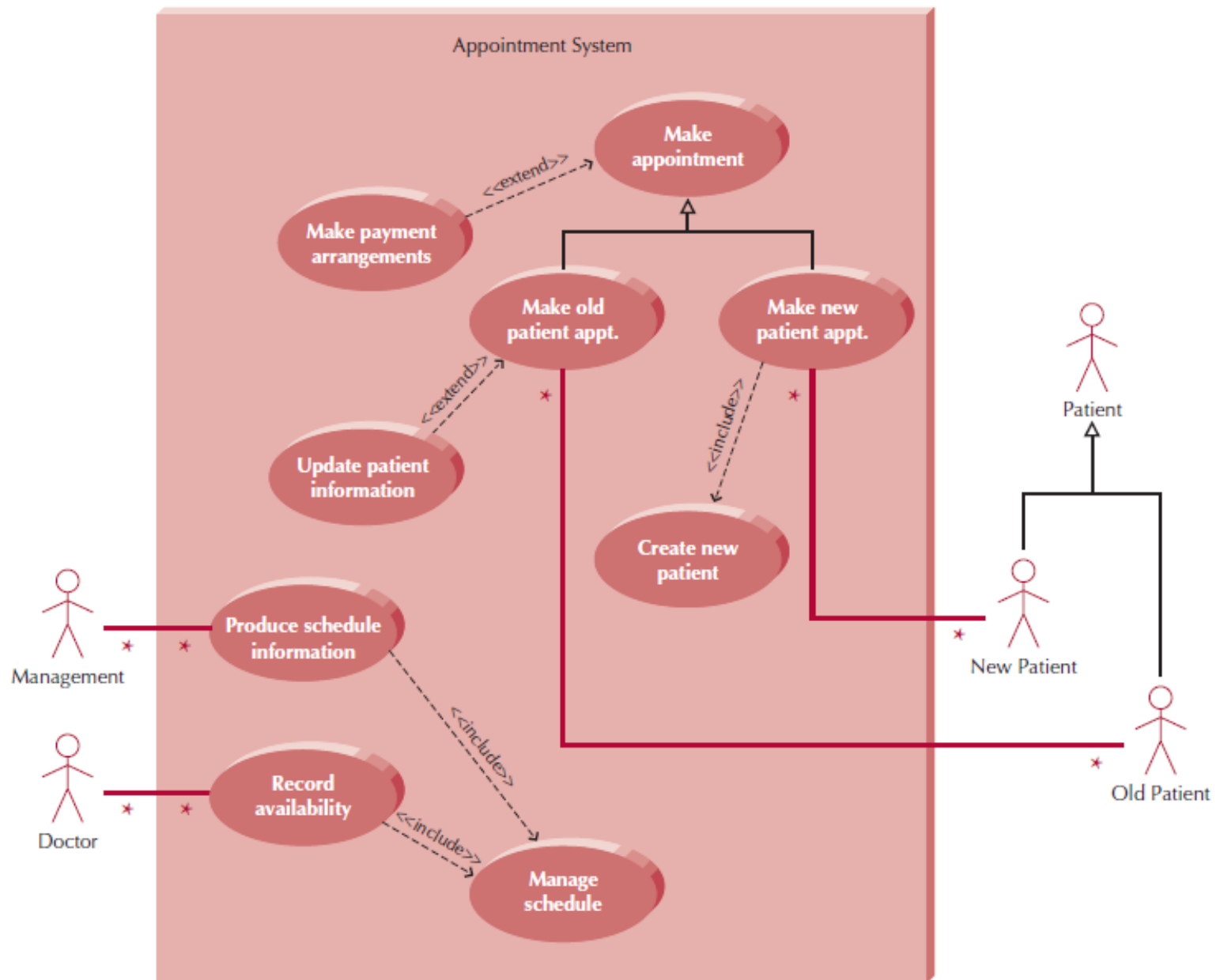


FIGURE 6-9 Extend and Include Relationships

Use Case Document

- A narrative document that describes the sequence of system events and the system responses originating from the initiating actors of the system.
- A use case tells a story of actors using a system
- A use-case is a sequence of actions a system performs that yields an observable result of value to a particular actor.

Terms and Concepts

- Actor:
 - something with behavior, such as a person, computer system, or organization, e.g. a cashier.
- Scenario:
 - specific sequence of actions and interactions between actors and the system under discussion, e.g. the scenario of successfully purchasing items with cash.
- Use case:
 - a collection of related success and failure scenarios that describe actors using a system to support a goal.

Scenarios

- Main success scenario:
 - The normal flow of processing
 - e.g., A customer arrives at a checkout with items to return. The cashier uses the POS system to record each returned item...
- Alternate scenarios:
 - If normal flow deviates, then the alternate course of action
 - e.g., If the credit authorization is reject, inform customer and ask for an alternative payment method. If item identifier not found in the system, notify the Cashier and suggest manual entry of the identifier code.

Things to remember!

- Choose the system boundary.
- Identify primary actors.
 - Those that have user goals fulfilled through using services of the system
- For each actor, identify their user goals.
- Tabulate findings in the Vision artifact.
- Define use cases that satisfy user goals; name them according to their goal.

Example: Fully dressed Use Case

- Use case UC1: Process Sale
- Primary Actor: Cashier
- Stakeholders and Interests:
 - -Cashier: Wants accurate and fast entry, no payment errors, ...
 - -Salesperson: Wants sales commissions updated.
- Preconditions: Cashier is identified and authenticated.
- Success Guarantee (Postconditions):
 - -Sale is saved. Tax correctly calculated.
- Main success scenario (or basic flow): [on next slide]
- Extensions (or alternative flows): [on next slide]
- Special requirements: Touch screen UI, ...
- Technology and Data Variations List:
 - -Identifier entered by bar code scanner,...
- Open issues: What are the tax law variations? ...

UC1: Process Sale

- **Main success scenario (or basic flow):**
 - The Customer arrives at a POS checkout with items to purchase.
 - The cashier records the identifier for each item. If there is more than one of the same item, the Cashier can enter the quantity as well.
 - The system determines the item price and adds the item information to the running sales transaction. The description and the price of the current item are presented.
 - On completion of item entry, the Cashier indicates to the POS system that item entry is complete.
 - The System calculates and presents the sale total.
 - The Cashier tells the customer the total.
 - The Customer gives a cash payment (“cash tendered”) possibly greater than the sale total.
- **Extensions (or alternative flows):**
 - If invalid identifier entered. Indicate error.
 - If customer didn’t have enough cash, cancel sales transaction.

Use Case Name: Make appointment		ID: 2	Importance Level: High
Primary Actor: Patient		Use Case Type: Detail, essential	
Stakeholders and Interests: Patient - wants to make, change, or cancel an appointment Doctor - wants to ensure patient's needs are met in a timely manner			
Brief Description: This use case describes how we make an appointment as well as changing or canceling an appointment.			
Trigger: Patient calls and asks for a new appointment or asks to cancel or change an existing appointment.			
Type: External			
Relationships: Association: Patient Include: Make Payment Arrangements Extend: Create New Patient Generalization:			
Normal Flow of Events: 1. The Patient contacts the office regarding an appointment. 2. The Patient provides the Receptionist with their name and address. 3. The Receptionist validates that the Patient exists in the Patient database. 4. The Receptionist executes the Make Payment Arrangements use case. 5. The Receptionist asks Patient if he or she would like to make a new appointment, cancel an existing appointment, or change an existing appointment. - If the patient wants to make a new appointment, the S-1: new appointment subflow is performed. - If the patient wants to cancel an existing appointment, the S-2: cancel appointment subflow is performed. - If the patient wants to change an existing appointment, the S-3: change appointment subflow is performed. 6. The Receptionist provides the results of the transaction to the Patient.			
Subflows: - S-1: New Appointment 1. The Receptionist asks the Patient for possible appointment times. 2. The Receptionist matches the Patient's desired appointment times with available dates and times and schedules the new appointment. - S-2: Cancel Appointment 1. The Receptionist asks the Patient for the old appointment time. 2. The Receptionist finds the current appointment in the appointment file and cancels it. - S-3: Change Appointment 1. The Receptionist performs the S-2: cancel appointment subflow. 2. The Receptionist performs the S-1: new appointment subflow.			
Alternate/Exceptional Flows: 3a: The Receptionist executes the Create New Patient use case. - S-1, 2a1: The Receptionist proposes some alternative appointment times based on what is available in the appointment schedule. - S-1, 2a2: The Patient chooses one of the proposed times or decides not to make an appointment.			

FIGURE 6-4 Use Case Description Example

Summary

- ANALYSIS PHASE (OBJECT ORIENTED DESIGN)
- Functional Modeling
 - Use case
 - Diagram
 - Description

